

P1028R4: SG14 `status_code` and standard `error` object

Document #: P1028R4
Date: 2022-10-28
Project: Programming Language C++
Library Evolution Working Group
Reply-to: Niall Douglas
<s_sourceforge@nedprod.com>

A proposal for the replacement, in new code, of the system header `<system_error>` with a substantially refactored and lighter weight design, which meets modern C++ design and implementation. This paper received the following vote at the May 2018 meeting of SG14: 8/2/1/0/0 (SF/WF/N/WA/SA).

A C++ 11 reference implementation of the proposed replacement can be found at <https://github.com/ned14/status-code>. Support for the proposed objects has been wired into Boost.Outcome [1] which has been shipping with the Boost C++ Libraries for three years.

The reference implementation has been found to work well on recent editions of GCC, clang and Microsoft Visual Studio, on x86, x64, ARM and AArch64. It has been quite popular with the C++ userbase, indeed there are two known complete re-implementations, one of which was described by [P2170] *Feedback on implementing the proposed `std::error` type*. This proposed design has shipped on every recent copy of Microsoft Windows and Apple iOS, and a fair chunk of Android devices. I believe it is amongst the best tested designs proposed for library standardisation in recent years.

This proposal is a much richer and more powerful framework than `<system_error>`, whilst remaining fully backwards compatible with it. Indeed, it can almost completely replace the dynamic exception mechanism with a fully deterministic alternative, and it has been proposed as the `std::error` implementation for [P0709] *Zero overhead deterministic exceptions* in [P1095] *Zero overhead deterministic failure*.

My apologies for the two years which have elapsed since R3. ‘Real life’ has intervened on my standards paper writing, and it is only due to my current client MayStreet London Stock Exchange Group allowing me a few work hours per month to work on standards papers that you see this R4 now and not yet more months from now. I hope that normal service will resume in 2023. I will not be at the upcoming Kona meeting, but I currently expect to be at the Issaquah meeting in early 2023.

Main change since R3:

- Added a new section reviewing design feedback from reimplementers.
- Many formerly non-`constexpr` member functions became `constexpr` thanks to C++ 20. Thanks to whomever suggested this for the reference implementation about a year ago.
- The only major design fix in the past two years was to add `status_code_domain::payload_info()`. This describes the alignment of erased payloads, so the correct bytes are copied when de-erasing a payload. New constructors have been added to erased status codes allowing construction from a compile-time unknown status code type, these will fail at runtime if the construction is incompatible.
- Added `quick_status_code_from_enum` which was suggested by a reimplementer.
- Semantics comparisons can now compare to a wider set of types than previously (i.e. we allow for implicit conversions).
- Added `status_error<void>::code()` to obtain access to the unknown status code from an unknown status code exception type.
- Added `posix_code::current()` and `win32_code::current()` to wrap the system's API for fetching the current platform error state.
- Added missing mixins for `std_error_code`, `posix_code`, `win32_code` etc.
- Added `http_status_code`.
- Reworked addendum about `result<T>` now that Boost.System has reimplemented the same facility.

Contents

1	Introduction	3
2	Impact on the Standard	5
3	Feedback from reimplementors of the proposed design	5
3.1	Jesse Towner	5
3.2	[P2170] <i>Feedback on implementing the proposed std::error type</i>	8
4	Proposed Design	9
4.1	<code>status_code_domain</code>	9
4.2	Traits	13
4.3	<code>status_code<void></code>	13
4.4	<code>detail::status_code_storage<DomainType></code>	15
4.5	<code>status_code<DomainType></code>	16
4.6	<code>status_code<erased<TRIVIALY_COPYABLE_OR_MOVE_BITCOPYING_TYPE>></code>	18
4.7	Status code comparisons	19
4.8	Exception types	21
4.9	Generic error coding	22
4.10	<code>errored_status_code<DomainType></code>	24
4.11	<code>errored_status_code<erased<TRIVIALY_COPYABLE_OR_MOVE_BITCOPYING_TYPE>></code>	26
4.12	Errored status code comparisons	28

4.13	Quick declaration of a new status code domain	30
4.14	OS specific and common codes	31
4.15	Erased system code, and proposed <code>std::error</code> object	33
4.16	iostream printing support	34
4.17	status code ptr	34
5	Design decisions, guidelines and rationale	35
5.1	Do not cause <code>#include <string></code>	35
5.2	All constexpr sourcing, construction and destruction	36
5.3	Header only libraries can now safely define custom code categories	36
5.4	No more <code>if(!ec)...</code>	37
5.5	No more filtering codes returned by system APIs	38
5.6	All comparisons between codes are now semantic, not literal	38
5.7	<code>std::error_condition</code> is removed entirely	38
5.8	<code>status_code</code> 's value type is set by its domain	39
5.9	<code>status_code<DomainType></code> is type erasable	39
5.10	More than one 'system' error coding domain: <code>system_code</code>	39
5.11	<code>std::errc</code> gets its own code domain <code>generic_code</code> , eliminating <code>std::error_condition</code>	40
6	Technical specifications	40
7	Frequently asked questions	40
7.1	Implied in this design is that code domains must do nothing in their constructor and destructors, as multiple instances are permitted and both must be trivial and constexpr. How then can dynamic per-domain initialisation be performed e.g. setting up at run time a table of localised message strings?	40
7.2	Move only <code>std::error</code> ?	41
8	Addendum: Boost's <code>result<T></code> type	41
9	Acknowledgements	45
10	References	45

1 Introduction

The `<system_error>` header entered the C++ standard in the C++ 11 standard, the idea for which having been split off from the Filesystem TS proposal into its own [N2066] proposal back in 2006. Despite its relative lack of direct usage by the C++ userbase, according to [2], `<system_error>` has become one of the most common internal dependencies for all other standard header files, frequently constituting up to 20% of all the tokens brought into the compiler by other standard header files e.g. `<array>`, `<complex>` or `<optional>`. In this sense, it is amongst the most popular system headers in the C++ standard library.

So why would anyone want to replace it? It unfortunately suffers from a number of design problems

only now apparent after twelve years of hindsight, which makes it low hanging fruit in the achievement of the ‘reduce compile time’ and ‘alternatives to complicated and/or error-prone features’ goals listed in [P2000] *Direction for ISO C++*. We, from Study Group 14 (the GameDev & Low Latency WG21 working group), listed many of these problems in [P0824], and after an extensive period of consultation with other stakeholders including the Boost C++ Libraries, we thence designed and implemented an improved substitute which does not have those problems. It is this improved, fully backwards compatible, design that we propose now.

This proposed library may be useful as the standardised implementation of the lightweight throwable `error` object as proposed by [P0709] *Zero-overhead deterministic exceptions: Throwing values*. It is [P0829] *Freestanding C++* compatible i.e. without dependency on any STL or language facility not usable on embedded systems.

An example of use:

```

1 std::system_code sc; // default constructs to empty
2 native_handle_type h = open_file(path, sc);
3 // Is the code a failure?
4 if(sc.failure())
5 {
6     // Do semantic comparison to test if this was a file not found failure
7     // This will match any system-specific error codes meaning a file not found
8     if(sc != std::errc::no_such_file_or_directory)
9     {
10         std::cerr << "FATAL: " << sc.message().c_str() << std::endl;
11         std::terminate();
12     }
13 }

```

The above is 100% portable code. Meanwhile, the implementation of `open_file()` might be these:

<pre> 1 // POSIX implementation 2 using native_handle_type = int; 3 native_handle_type open_file(const char *path, 4 std::system_code &sc) noexcept 5 { 6 sc.clear(); // clears to empty 7 native_handle_type h = ::open(path, O_RDONLY); 8 if(-1 == h) 9 { 10 // posix_code type erases into system_code 11 sc = std::posix_code(errno); 12 } 13 return h; 14 } </pre>	<pre> 1 // Microsoft Windows implementation 2 using native_handle_type = HANDLE; 3 native_handle_type open_file(const wchar_t *path, 4 std::system_code &sc) noexcept 5 { 6 sc.clear(); // clears to empty 7 native_handle_type h = CreateFile(path, 8 GENERIC_READ, 9 FILE_SHARE_READ FILE_SHARE_WRITE 10 FILE_SHARE_DELETE, 11 nullptr, 12 OPEN_EXISTING, 13 FILE_ATTRIBUTE_NORMAL, 14 nullptr); 15 if(INVALID_HANDLE_VALUE == h) 16 { 17 // win32_code type erases into system_code 18 sc = std::win32_code(GetLastError()); 19 } 20 return h; 21 } </pre>
---	--

2 Impact on the Standard

The proposed library is a pure-library solution.

It would be great if the object could be bit relocatable e.g. via [P1029] *move = bitcopies* or equivalent, but I suspect that support for those won't be done by EWG in time. Some compilers e.g. clang have proprietary attributes which implement bitcopying moves.

3 Feedback from reimplementors of the proposed design

3.1 Jesse Towner

Quoted with permission from Jesse Towner:

I'd like to give you some feedback on how we've been using `status_code` and `error` in a few large projects over the past couple of years. We've had a lot of success with the library, but there have been a few rough edges and a major pain point, but more on that below as well as my solution to it.

Now, just for some background, I've implemented our own independent version of the status-code library directly from P1028, and did so for a couple of reasons. First, for better integration into our own proprietary low-level C++ extensions library and second because we're currently using C++17 as a baseline and that naturally allows for a much more concise implementation. Furthermore, doing so allowed me to vet your design and give you all of that feedback a while ago as well, so there's also that, hehe. Anyway, you might be happy to know that we're using this in Microsoft Solitaire Collection, which of course comes pre-installed on Windows 10 these days and has wide adoption in both the Apple and Google Stores. You can probably imagine the size of the install base. So if anyone is still doubting that the status-code library as proposed isn't in wide use, you can always give them that, hehe.

So jumping right into things, we started off by using type-erased error objects for propagating errors in an exceptionless manner across threads with our own multi-platform concurrency library that has futures & promises, senders & receivers and executors. We opted to use `std::expected` to package return value and errors together, since at the time it seemed fairly mature in its design. All of `std::expecteds` copy constructors and copy-assignment operators automatically get excluded from the overload set when using a move-only Error object, and then all that needed to be done was to specialize `std::bad_expected_access` for both `status_code<erased<ErasedType>>` and `error` so that it calls `.clone()` to store a copy of the underlying Error. If we were to do it all over again, we might consider using `result<T>` but we will probably hold off on changing anything until LEWG makes further decisions on P1028 and P0709. So if they don't like neither P0709 nor `result<T>`, in case you haven't thought of it already – although I bet you probably have – you could as a last resort try pitching `std::expected` with move-only `status_codes` using the above technique to allow P1028 to go through, perhaps

just making `std::result<T>` be a template alias for `std::expected<T, std::error>`. But I digress.

Moving on, I think the core part of the status-code library is a little complex and you certainly have already heard complaints along those lines. But after implementing it myself, I think that complexity is fully justified given what the library does for us. It really enables a rich and powerful way of working with various different types of error codes in a more-or-less orthogonal and extensible manner. It address all of the issues with `<system_error>`. For all of the builtin system-level error codes, the library does a great job. And for users of the library, things are more or less straightforward and easy to comprehend when working directly with a `status_code` or error object, especially once they understand how semantic equivalence testing works and that you can have equivalence classes of status codes in the same way that you can with exceptions.

The big problem, and this was the major point of pain for us I was talking about earlier, is that's really really difficult to teach everyone how to effectively write concrete `status_code_domains` for custom enum-based status codes. The reasons for this I imagine are kind of related to the same reasons its often hard to get everyone to write exception-safe code all of the time, which is why we originally chose not to use exception handling in the first place. The intricacies of error handling sadly is not an area of interest for most people, and when you're on a deadline and needing to finish off implementing end-user facing features for the next milestone, things like setting up and utilizing `status_code_domains` properly or writing exception-safe code all end up low on the priority list.

Having to write one to two hundred lines of code to create a `status_code_domain` each time you want to allow an enum to be used with the status-code system just doesn't scale well. On the other hand, when you look at exceptions in C++, the one nice thing about exceptions is that it's a really simple process to create a new exception class. You merely derive from `std::exception` or another child class of `std::exception` that already exists and you're off to the races. If you need to give the exception some state and a custom error message, you add some member variables and override `what()`. Easy. It would be really nice if creating custom `status_code_domains` for enums, or indeed any trivial or move-relocatable class type, would be just as simple. I believe status code domains need to be invisible most of the time, and yet something that you can get at when needed and have it all just work.

Now initially, I myself played this off as not a big issue. Mostly because at the time I couldn't think of a better way of doing things. But as the complaints continued to roll in and as the situation worsened, I realized something needed to be done about it or we'd be in trouble. I won't go into the precise details, but even even for the C++ experts, having to set up a new `status_code_domain` was becoming an exercise in writing boilerplate, when we'd rather be working on more important things. When you have multiple dozens of custom enum-based status codes all throughout your code base for your various application level components and services, it's a real chore to maintain all of that.

[... Jesse goes on to propose a solution which ended up becoming proposed

`quick_status_code_from_enum` below ...]

My thanks to Jesse for taking the time to send such a detailed piece of design feedback and suggestions for improvement, which were incorporated since R3 of this paper.

An example of `quick_status_code_from_enum` in use:

```
1 namespace another_namespace
2 {
3     // This is some custom enum code
4     enum class AnotherCode : size_t
5     {
6         success1,
7         goaway,
8         success2,
9         error2
10    };
11 } // namespace another_namespace
12
13
14 /* Rather than copy and paste the extensive boilerplate to declare a custom status code domain,
15 have the compiler generate it via metaprogramming. We simply supply the bits to poke into the
16 boilerplate generation.
17
18 'quick_status_code_from_enum_defaults' sets sensible defaults so you only need to override
19 the specific bits you want to customise.
20 */
21 template <>
22 struct quick_status_code_from_enum<another_namespace::AnotherCode>
23     : quick_status_code_from_enum_defaults<another_namespace::AnotherCode>
24 {
25     // Text name of the enum
26     static constexpr const auto domain_name = "Another Code";
27
28     // Unique UUID for the enum. PLEASE use https://www.random.org/cgi-bin/randbyte?nbytes=16&format=h
29     static constexpr const auto domain_uuid = "{be201f65-3962-dd0e-1266-a72e63776a42}";
30
31     // Map of each enum value to its text string, and list of semantically equivalent errc's
32     static const std::initializer_list<mapping> &value_mappings()
33     {
34         static const std::initializer_list<mapping> v = {
35
36             // Format is: { enum value, "string representation", { list of errc mappings ... } }
37             {another_namespace::AnotherCode::success1, "Success 1", {errc::success}}, //
38             {another_namespace::AnotherCode::goaway, "Go away", {errc::permission_denied}}, //
39             {another_namespace::AnotherCode::success2, "Success 2", {errc::success}}, //
40             {another_namespace::AnotherCode::error2, "Error 2", {}}, //
41
42         };
43         return v;
44     }
45
46     // Completely optional definition of mixin for the status code synthesised from 'Enum'.
47     // It can be omitted.
48     template <class Base> struct mixin : Base
49     {
```

```

50     using Base::Base;
51     constexpr int custom_method() const { return int(this->value()); }
52 };
53 };
54
55 namespace another_namespace
56 {
57     // ADL discovered, must be in same namespace as AnotherCode
58     constexpr inline
59     quick_status_code_from_enum_code<another_namespace::AnotherCode>
60     status_code(AnotherCode c)
61     {
62         // Because we customised quick_status_code_from_enum above, status_code
63         // knows now how to implicitly construct from a AnotherCode
64         return c;
65     }
66 } // namespace another_namespace
67
68
69 // status_code<erased<intptr_t>> will now construct from AnotherCode
70 system_code v1(another_namespace::AnotherCode::error2);
71
72 // ADL discovered convenience factory function is another option
73 // Note the mixed in 'custom_method()' to the status code type
74 constexpr auto v2 = status_code(another_namespace::AnotherCode::error2);
75 assert(v2.value() == another_namespace::AnotherCode::error2);
76 assert(v2.custom_method() == 3);

```

3.2 [P2170] *Feedback on implementing the proposed `std::error` type*

P2170 made a number of alternative design suggestions:

1. Proposed `std::error` ought to be copy constructible as well as move constructible, thus asking the domain to perform a copy per `std::error` copy.
2. `std::error_code` ought to be convertible into `std::error`.
3. Proposed `std::generic_code` ought to represent a different enum to `std::errc`.
4. Remove everything outside `std::error` and `std::error_domain` (with four domains supplied out of the box) as being superfluous to need.
5. Semantic comparison ought to be simpler than that proposed in this paper.
6. The unique id ought to be 128 bit, not 64 bit.

My reply to those design points:

1. P1028 provides `.clone()` if you want to copy a status code. It does not produce good codegen, because it requires asking the domain to make the copy (and it may throw an exception refusing or rejecting the request). Insisting on move-only semantics for `std::error` makes for *much* better codegen because the compiler can collapse inlined TRY operations, you can see an example of this at <https://godbolt.org/z/qdnY78cxn>. I would also point out that if you

really want a copy constructible `error`, it's trivial to build your own on top of `status_code`. Or, just use `std::error_code`, it's not going anywhere.

2. Proposed `std::error` already implicitly constructs from `std::error_code`, with full preservation of the error code's `std::error_category` i.e. to be clear here, the **original** `std::error_category` is used for semantic comparisons to status codes if the input error category does not map onto a status code domain (i.e. is a custom third party defined error category).
3. I would suggest that the enum values in `std::errc` can be extended with new values at any time by the committee.
4. It is tempting to think that one just needs a `std::error` and the minimum possible additional. However, in my opinion, as soon as your needs exceed bare minimum, you'll then find yourself straightjacketed by the limited design. The design proposed by this paper has stood up very well over the past three years of production use by multiple people not just this author, with only a few minor design issues discovered and fixed in that time (alignment of erased codes was the big one).
5. One of the specific reasons that semantic comparisons are more complex than would be otherwise necessary is to preserve semantic comparison compatibility with `std::error_category` such that status codes can be compared to error codes using an arbitrary third party supplied error category instance. I think this backwards compatibility worth preserving, even if semantic comparison logic is then slightly more complex.
6. I would refer to [P0824] *Summary of SG14 discussion on <system_error>* where the statistical chance of collision in a properly random 64 bit unsigned integer is considered to be very low. As an example, you would need 190 unique status code domains in a process to have the same chance of collision as a bit flipping on your entire hard drive.

4 Proposed Design

4.1 `status_code_domain`

```
1  /*! The main workhorse of the system_error2 library, can be typed
2  ('status_code<DomainType>'), erased-immutable ('status_code<void>') or
3  erased-mutable ('status_code<erased<T>>').
4
5  Be careful of placing these into containers! Equality and inequality operators are
6  *semantic* not exact. Therefore two distinct items will test true! To help prevent
7  surprise on this, 'operator<' and 'std::hash<>' are NOT implemented in order to
8  trap potential incorrectness. Define your own custom comparison functions for your
9  container which perform exact comparisons.
10 */
11 template <class DomainType> class status_code;
12
13 class _generic_code_domain;
14
15 /*! The generic code is a status code with the generic code domain, which is that of 'errc' (POSIX).
16 using generic_code = status_code<_generic_code_domain>;
```

```

1  /*! Abstract base class for a coding domain of a status code.
2  */
3  class status_code_domain
4  {
5      template <class DomainType> friend class status_code;
6      template <class StatusCode> friend class indirecting_domain;
7
8  public:
9      ///! Type of the unique id for this domain.
10     using unique_id_type = unsigned long long;
11
12     /*! (Potentially thread safe) Reference to a message string.
13
14     Be aware that you cannot add payload to implementations of this class.
15     You get exactly the 'void *[3]' array to keep state, this is usually
16     sufficient for a 'std::shared_ptr<>' or a 'std::string'.
17
18     You can install a handler to be called when this object is copied,
19     moved and destructed. This takes the form of a C function pointer.
20     */
21     class string_ref
22     {
23     public:
24         ///! The value type
25         using value_type = const char;
26         ///! The size type
27         using size_type = size_t;
28         ///! The pointer type
29         using pointer = const char *;
30         ///! The const pointer type
31         using const_pointer = const char *;
32         ///! The iterator type
33         using iterator = const char *;
34         ///! The const iterator type
35         using const_iterator = const char *;
36
37     protected:
38         ///! The operation occurring
39         enum class _thunk_op
40         {
41             copy,
42             move,
43             destruct
44         };
45         ///! The prototype of the handler function. Copies can throw, moves and destructs cannot.
46         using _thunk_spec = void (*)(string_ref *dest, const string_ref *src, _thunk_op op);
47
48         ///! Pointers to beginning and end of character range
49         pointer _begin{}, _end{};
50
51         ///! Three 'void*' of state
52         void *_state[3]{}; // at least the size of a shared_ptr
53
54         ///! Handler for when operations occur
55         const _thunk_spec _thunk{nullptr};

```

```

56
57     constexpr explicit string_ref(_thunk_spec thunk) noexcept;
58
59 public:
60     //! Construct from a C string literal
61     constexpr explicit string_ref(const char *str, size_type len = static_cast<size_type>(-1),
62                                   void *state0 = nullptr, void *state1 = nullptr,
63                                   void *state2 = nullptr, _thunk_spec thunk = nullptr) noexcept;
64     //! Copy construct the derived implementation.
65     constexpr string_ref(const string_ref &o);
66     //! Move construct the derived implementation.
67     constexpr string_ref(string_ref &o) noexcept;
68     //! Copy assignment
69     constexpr string_ref &operator=(const string_ref &o);
70     //! Move assignment
71     constexpr string_ref &operator=(string_ref &o) noexcept;
72     //! Destruction
73     constexpr ~string_ref();
74
75     //! Returns whether the reference is empty or not
76     [[nodiscard]] constexpr bool empty() const noexcept;
77     //! Returns the size of the string
78     constexpr size_type size() const noexcept;
79     //! Returns a null terminated C string
80     constexpr const_pointer c_str() const noexcept;
81     //! Returns a null terminated C string
82     constexpr const_pointer data() const noexcept;
83     //! Returns the beginning of the string
84     constexpr iterator begin() noexcept;
85     //! Returns the beginning of the string
86     constexpr const_iterator begin() const noexcept;
87     //! Returns the beginning of the string
88     constexpr const_iterator cbegin() const noexcept;
89     //! Returns the end of the string
90     constexpr iterator end() noexcept;
91     //! Returns the end of the string
92     constexpr const_iterator end() const noexcept;
93     //! Returns the end of the string
94     constexpr const_iterator cend() const noexcept;
95 };
96
97 /*! A reference counted, threadsafe reference to a message string.
98 */
99 class atomic_refcounted_string_ref : public string_ref
100 {
101     struct _allocated_msg
102     {
103         mutable std::atomic<unsigned> count;
104     };
105     _allocated_msg *_msg() noexcept;
106     const _allocated_msg *_msg() const noexcept;
107
108     static void _refcounted_string_thunk(string_ref *_dest, const string_ref *_src, _thunk_op op)
109         noexcept;
110 public:

```

```

111     ///! Construct from a C string literal allocated using 'malloc()'.
112     explicit atomic_refcounted_string_ref(const char *str, size_type len = static_cast<size_type>(-1),
113                                           void *state1 = nullptr, void *state2 = nullptr) noexcept;
114 };
115
116 private:
117     unique_id_type _id;
118
119 protected:
120     ///! Use [https://www.random.org/cgi-bin/randbyte?nbytes=8&format=h](https://www.random.org/cgi-bin/
121         randbyte?nbytes=8&format=h) to get a random 64 bit id.
122     Do NOT make up your own value. Do NOT use zero.
123     */
124     constexpr explicit status_code_domain(unique_id_type id) noexcept;
125     ///! No public copying at type erased level
126     status_code_domain(const status_code_domain &) = default;
127     ///! No public moving at type erased level
128     status_code_domain(status_code_domain &&) = default;
129     ///! No public assignment at type erased level
130     status_code_domain &operator=(const status_code_domain &) = default;
131     ///! No public assignment at type erased level
132     status_code_domain &operator=(status_code_domain &&) = default;
133     ///! No public destruction at type erased level
134     ~status_code_domain() = default;
135
136 public:
137     ///! True if the unique ids match.
138     constexpr bool operator==(const status_code_domain &) const noexcept;
139     ///! True if the unique ids do not match.
140     constexpr bool operator!=(const status_code_domain &) const noexcept;
141     ///! True if this unique is lower than the other's unique id.
142     constexpr bool operator<(const status_code_domain &) const noexcept;
143
144     ///! Returns the unique id used to identify identical category instances.
145     constexpr unique_id_type id() const noexcept;
146     ///! Name of this category.
147     constexpr virtual string_ref name() const noexcept = 0;
148     ///! Information about the payload of the code for this domain
149     struct payload_info_t
150     {
151         size_t payload_size;      ///!< The payload size in bytes
152         size_t total_size;        ///!< The total status code size in bytes (includes domain pointer and
153             mixins state)
154         size_t total_alignment;    ///!< The total status code alignment in bytes
155
156         payload_info_t() = default;
157         constexpr payload_info_t(size_t _payload_size, size_t _total_size, size_t _total_alignment);
158     };
159     ///! Information about this domain's payload
160     constexpr virtual payload_info_t payload_info() const noexcept = 0;
161
162 protected:
163     ///! True if code means failure.
164     constexpr virtual bool _do_failure(const status_code<void> &code) const noexcept = 0;
165     ///! True if code is (potentially non-transitively) equivalent to another code in another domain.
166     constexpr virtual bool _do_equivalent(const status_code<void> &code1, const status_code<void> &code2

```

```

    ) const noexcept = 0;
165  ///! Returns the generic code closest to this code, if any.
166  constexpr virtual generic_code _generic_code(const status_code<void> &code) const noexcept = 0;
167  ///! Return a reference to a string textually representing a code.
168  constexpr virtual string_ref _do_message(const status_code<void> &code) const noexcept = 0;
169  ///! Throw a code as a C++ exception.
170  [[noreturn]] constexpr virtual void _do_throw_exception(const status_code<void> &code) const = 0;
171  };

```

4.2 Traits

```

1  ///! Namespace for user injected mixins
2  namespace mixins
3  {
4      template <class Base, class T> struct mixin : public Base
5      {
6          using Base::Base;
7      };
8  }
9
10  /*! A tag for an erased value type for 'status_code<D>'.
11  Available only if 'ErasedType' satisfies 'traits::is_move_relocating<ErasedType>::value'.
12  */
13  template <class ErasedType>
14  requires(traits::is_move_relocating<ErasedType>::value)
15  struct erased
16  {
17      using value_type = ErasedType;
18  };
19
20  template <class Enum> struct quick_status_code_from_enum;
21
22  ///! Trait returning true if the type is a status code.
23  template <class T> struct is_status_code;
24  template <class T> static constexpr bool is_status_code_v;

```

4.3 status_code<void>

```

1  /*! A type erased lightweight status code reflecting empty, success, or failure.
2  Differs from 'status_code<erased<>>' by being always available irrespective of
3  the domain's value type, but cannot be copied, moved, nor destructed. Thus one
4  always passes this around by const lvalue reference.
5  */
6  template <> class status_code<void>
7  {
8      template <class T> friend class status_code;
9
10  public:
11      ///! The type of the domain.
12      using domain_type = void;
13      ///! The type of the status code.

```

```

14     using value_type = void;
15     /// The type of a reference to a message string.
16     using string_ref = typename status_code_domain::string_ref;
17
18 protected:
19     const status_code_domain *_domain{nullptr};
20
21 protected:
22     ///! No default construction at type erased level
23     status_code() = default;
24     ///! No public copying at type erased level
25     status_code(const status_code &) = default;
26     ///! No public moving at type erased level
27     status_code(status_code &&) = default;
28     ///! No public assignment at type erased level
29     status_code &operator=(const status_code &) = default;
30     ///! No public assignment at type erased level
31     status_code &operator=(status_code &&) = default;
32     ///! No public destruction at type erased level
33     ~status_code() = default;
34
35     ///! Used to construct a non-empty type erased status code
36     constexpr explicit status_code(const status_code_domain *v) noexcept;
37
38 public:
39     ///! Return the status code domain.
40     constexpr const status_code_domain &domain() const noexcept;
41     ///! True if the status code is empty.
42     [[nodiscard]] constexpr bool empty() const noexcept;
43
44     ///! Return a reference to a string textually representing a code.
45     constexpr string_ref message() const noexcept;
46     ///! True if code means success.
47     constexpr bool success() const noexcept;
48     ///! True if code means failure.
49     constexpr bool failure() const noexcept;
50
51     /*! True if code is strictly (and potentially non-transitively) semantically equivalent to
52     another code in another domain.
53
54     Note that usually non-semantic i.e. pure value comparison is used when the other
55     status code has the same domain. As 'equivalent()' will try mapping to generic code,
56     this usually captures when two codes have the same semantic meaning in 'equivalent()'.
57     */
58     template <class T> constexpr bool strictly_equivalent(const status_code<T> &o) const noexcept;
59
60     /*! True if code is equivalent, by any means, to another code in another domain
61     (guaranteed transitive).
62
63     Firstly 'strictly_equivalent()' is run in both directions. If neither succeeds, each domain
64     is asked for the equivalent generic code and those are compared.
65     */
66     template <class T> constexpr bool equivalent(const status_code<T> &o) const noexcept;
67
68     ///! Throw a code as a C++ exception.
69     [[noreturn]] constexpr void throw_exception() const;

```

```
70 };
```

4.4 detail::status_code_storage<DomainType>

It is highly unusual for items in a detail namespace to be proposed for standardisation. However it was felt that until a judgement is taken on mixins, it was best to retain the structure of the reference library implementation.

```
1 namespace detail
2 {
3     template <class DomainType> struct get_domain_value_type
4     {
5         using domain_type = DomainType;
6         using value_type = typename domain_type::value_type;
7     };
8     template <class ErasedType> struct get_domain_value_type<erased<ErasedType>>
9     {
10         using domain_type = status_code_domain;
11         using value_type = ErasedType;
12     };
13     template <class DomainType> class status_code_storage : public status_code<void>
14     {
15         static_assert(!std::is_void<DomainType>::value, "status_code_storage<void> should never occur!");
16     public:
17         ///! The type of the domain.
18         using domain_type = typename get_domain_value_type<DomainType>::domain_type;
19         ///! The type of the status code.
20         using value_type = typename get_domain_value_type<DomainType>::value_type;
21         ///! The type of a reference to a message string.
22         using string_ref = typename domain_type::string_ref;
23
24         ///! Return the status code domain.
25         constexpr const domain_type &domain() const noexcept;
26
27         ///! Reset the code to empty.
28         constexpr void clear() noexcept;
29
30         ///! Return a reference to the 'value_type'.
31         constexpr value_type &value() & noexcept;
32         ///! Return a reference to the 'value_type'.
33         constexpr value_type &&value() && noexcept;
34         ///! Return a reference to the 'value_type'.
35         constexpr const value_type &value() const &noexcept;
36         ///! Return a reference to the 'value_type'.
37         constexpr const value_type &&value() const &&noexcept;
38
39     protected:
40         status_code_storage() = default;
41         status_code_storage(const status_code_storage &) = default;
42         status_code_storage(status_code_storage &&) = default;
43         status_code_storage &operator=(const status_code_storage &) = default;
44         status_code_storage &operator=(status_code_storage &&) = default;
45         ~status_code_storage() = default;
```

```

46     value_type _value{};
47     struct _value_type_constructor { };
48     template <class... Args>
49     constexpr status_code_storage(_value_type_constructor /*unused*/, const status_code_domain *v,
50         Args &&... args);
51 };
52 } // namespace detail

```

4.5 status_code<DomainType>

```

1  /*! A lightweight, typed, status code reflecting empty, success, or failure.
2  This is the main workhorse of the system_error2 library.
3
4  An ADL discovered helper function 'make_status_code(T, Args...)' is looked up by one
5  of the constructors. If it is found, and it generates a status code compatible with this
6  status code, implicit construction is made available.
7
8  You may mix in custom member functions and member function overrides by injecting a specialisation of
9  'mixins::mixin<Base, YourDomainType>'. Your mixin must inherit from 'Base'.
10 */
11 template <class DomainType>
12 requires(
13     (!std::is_default_constructible<typename DomainType::value_type>::value
14      || std::is_nothrow_default_constructible<typename DomainType::value_type>::value)
15     && (!std::is_move_constructible<typename DomainType::value_type>::value
16        || std::is_nothrow_move_constructible<typename DomainType::value_type>::value)
17     && std::is_nothrow_destructible<typename DomainType::value_type>::value
18 )
19 class status_code : public mixins::mixin<detail::status_code_storage<DomainType>, DomainType>
20 {
21     template <class T> friend class status_code;
22
23 public:
24     ///! The type of the domain.
25     using domain_type = DomainType;
26     ///! The type of the status code.
27     using value_type = typename domain_type::value_type;
28     ///! The type of a reference to a message string.
29     using string_ref = typename domain_type::string_ref;
30
31 public:
32     ///! Default construction to empty
33     status_code() = default;
34     ///! Copy constructor
35     status_code(const status_code &) = default;
36     ///! Move constructor
37     status_code(status_code &&) = default;
38     ///! Copy assignment
39     status_code &operator=(const status_code &) = default;
40     ///! Move assignment
41     status_code &operator=(status_code &&) = default;
42     ~status_code() = default;
43

```



```

44  //! Return a copy of the code.
45  constexpr status_code clone() const;
46
47  //! Implicit construction from any type where an ADL discovered
48  //! 'make_status_code(T, Args ...)' returns a 'status_code'.
49  template <class T, class... Args, class MakeStatusCodeResult =
50      typename detail::safe_get_make_status_code_result<T, Args...>::type> // Safe ADL lookup of
        make_status_code(), returns void if not found
51  requires(!std::is_same<typename std::decay<T>::type, status_code>::value // not copy/move of self
52      && !std::is_same<typename std::decay<T>::type, in_place_t>::value // not in_place_t
53      && is_status_code<MakeStatusCodeResult>::value // ADL makes a status
        code
54      && std::is_constructible<status_code, MakeStatusCodeResult>::value)) // ADLed status code
        is compatible
55  )
56  constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::declval<T>(),
        std::declval<Args>()...)));
57
58  //! Implicit construction from any 'quick_status_code_from_enum<Enum>' enumerated type.
59  template<class Enum, class QuickStatusCodeType
60      = typename quick_status_code_from_enum<Enum>::code_type> // Enumeration has been activated
61  requires(std::is_constructible<status_code, QuickStatusCodeType>::value) // Its status code is
        compatible
62  constexpr status_code(Enum &&v) noexcept(std::is_nothrow_constructible<status_code,
        QuickStatusCodeType>::value);
63
64  //! Explicit in-place construction.
65  template <class... Args>
66  constexpr explicit status_code(in_place_t /*unused */, Args &&... args) noexcept(std::
        is_nothrow_constructible<value_type, Args &&...>::value);
67
68  //! Explicit in-place construction from initialiser list.
69  template <class T, class... Args>
70  constexpr explicit status_code(in_place_t /*unused */, std::initializer_list<T> il, Args &&... args)
        noexcept(std::is_nothrow_constructible<value_type, std::initializer_list<T>, Args &&...>::
        value);
71
72  //! Explicit copy construction from a 'value_type'.
73  constexpr explicit status_code(const value_type &v) noexcept(std::is_nothrow_copy_constructible<
        value_type>::value);
74
75  //! Explicit move construction from a 'value_type'.
76  constexpr explicit status_code(value_type &&v) noexcept(std::is_nothrow_move_constructible<
        value_type>::value);
77
78  /*! Explicit construction from an erased status code. Available only if
79  'value_type' is trivially destructible and 'sizeof(status_code) <= sizeof(status_code<erased<>>)'.
80  Does not check if domains are equal.
81  */
82  template <class ErasedType>
83  requires(detail::domain_value_type_erasure_is_safe<domain_type, erased<ErasedType>>::value)
84  constexpr explicit status_code(const status_code<erased<ErasedType>> &v) noexcept(std::
        is_nothrow_copy_constructible<value_type>::value);
85
86  //! Return a reference to a string textually representing a code.
87  constexpr string_ref message() const noexcept;

```

```
88 };
```

4.6 `status_code<erased<TRIVIALY_COPYABLE_OR_MOVE_BITCOPYING_TYPE>>`

```
1  /*! Type erased, move-only status_code, unlike 'status_code<void>' which cannot be moved nor
2  destroyed. Available only if 'erased<>' is available, which is when the domain's type is trivially
3  copyable or is move bitcopying, and if the size of the domain's typed error code is less than
4  or equal to this erased error code. Copy construction is disabled, but if you want a copy call
5  '.clone()'.
6
7  An ADL discovered helper function 'make_status_code(T, Args...)' is looked up by one of the
8  constructors. If it is found, and it generates a status code compatible with this status code,
9  implicit construction is made available.
10 */
11 template <class ErasedType> class status_code<erased<ErasedType>>
12 : public mixins::mixin<detail::status_code_storage<erased<ErasedType>>, erased<ErasedType>>
13 {
14     template <class T> friend class status_code;
15
16 public:
17     ///! The type of the domain (void, as it is erased).
18     using domain_type = void;
19     ///! The type of the erased status code.
20     using value_type = ErasedType;
21     ///! The type of a reference to a message string.
22     using string_ref = typename _status_code<void>::string_ref;
23
24 public:
25     ///! Default construction to empty
26     status_code() = default;
27     ///! Copy constructor
28     status_code(const status_code &) = delete;
29     ///! Move constructor
30     status_code(status_code &&) = default;
31     ///! Copy assignment
32     status_code &operator=(const status_code &) = delete;
33     ///! Move assignment
34     status_code &operator=(status_code &&) = default;
35     constexpr ~status_code();
36
37     ///! Return a copy of the erased code by asking the domain to perform the erased copy.
38     constexpr status_code clone() const;
39
40     ///! Implicit copy construction from any other status code if its value type is
41     ///! trivially copyable, it would fit into our storage, and it is not an erased
42     ///! status code.
43     template <class DomainType>
44     requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value
45              && !detail::is_erased_status_code<status_code<typename std::decay<DomainType>::type>>::value
46              )
47     constexpr status_code(const status_code<DomainType> &v) noexcept;
48
49     ///! Implicit move construction from any other status code if its value type is trivially copyable
50     ///! or move relocating and it would fit into our storage
```

```

50 template <class DomainType>
51 requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value)
52 constexpr status_code(status_code<DomainType> &&v) noexcept;
53
54 //! Implicit construction from any type where an ADL discovered 'make_status_code(T, Args ...)'
55 //! returns a 'status_code'.
56 template <class T, class... Args,
57           class MakeStatusCodeResult =
58             typename detail::safe_get_make_status_code_result<T, Args...>::type> // Safe ADL lookup of
59             make_status_code(), returns void if not found
60 requires(!std::is_same<typename std::decay<T>::type, status_code>::value // not copy/move of self
61           && !std::is_same<typename std::decay<T>::type, value_type>::value // not copy/move of
62           value type
63           && is_status_code<MakeStatusCodeResult>::value // ADL makes a status
64           code
65           && std::is_constructible<status_code, MakeStatusCodeResult>::value)) // ADLed status code
66           is compatible
67 )
68 constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::declval<T>()),
69                               std::declval<Args>()...));
70
71
72 //! Implicit construction from any 'quick_status_code_from_enum<Enum>' enumerated type.
73 template<class Enum,
74          class QuickStatusCodeType = typename quick_status_code_from_enum<Enum>::code_type>
75           // Enumeration has been activated
76 requires(std::is_constructible<status_code, QuickStatusCodeType>::value) // Its status code is
77           compatible
78 constexpr status_code(Enum &&v) noexcept(std::is_nothrow_constructible<status_code,
79                               QuickStatusCodeType>::value);
80
81
82 //! Explicit copy construction from an unknown status code. Note that this will throw
83 //! an exception if its value type is not trivially copyable or would not
84 //! fit into our storage or the source domain's '_do_erased_copy()' refused the copy.
85 //! This function is not present if C++ exceptions are globally disabled.
86 explicit constexpr status_code(const status_code<void> &v);
87
88
89 //! Tagged copy construction from an unknown status code. Note that this will be empty
90 //! if its value type is not trivially copyable or would not fit into our
91 //! storage or the source domain's '_do_erased_copy()' refused the copy.
92 constexpr status_code(std::nothrow_t, const status_code<void> &v) noexcept
93 };

```

4.7 Status code comparisons

```

1  //! True if the status code's are semantically equal via 'equivalent()'.
2  template <class DomainType1, class DomainType2>
3  constexpr bool operator==(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
4  noexcept;
5
6  //! True if the status code's are not semantically equal via 'equivalent()'.
7  template <class DomainType1, class DomainType2>
8  constexpr bool operator!=(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
9  noexcept;

```

```

9  //! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
10 template<class DomainType1, class T, class MakeStatusCodeResult
11     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
12 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
13 constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
14
15 //! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
16 template<class DomainType1, class T, class MakeStatusCodeResult
17     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
18 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
19 constexpr bool operator==(const T &a, const status_code<DomainType1> &b);
20
21 //! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)'.
22 template<class DomainType1, class T, class MakeStatusCodeResult
23     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
24 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
25 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
26
27 //! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)'.
28 template<class DomainType1, class T, class MakeStatusCodeResult
29     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
30 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
31 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);
32
33 //! True if the status code's are semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(b)'.
34 template <class DomainType1, class T, class QuickStatusCodeType
35     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
36 >
37 constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
38
39 //! True if the status code's are semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(a)'.
40 template <class DomainType1, class T, class QuickStatusCodeType
41     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
42 >
43 constexpr bool operator==(const T &a, const status_code<DomainType1> &b);
44
45 //! True if the status code's are not semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(b)'.
46 template <class DomainType1, class T, class QuickStatusCodeType
47     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
48 >
49 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
50
51 //! True if the status code's are not semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(a)'.
52 template <class DomainType1, class T, class QuickStatusCodeType

```

```

53     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
54 >
55 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);

```

4.8 Exception types

```

1  /*! Exception type representing a thrown status_code
2  */
3  template <class DomainType> class status_error;
4
5  /*! The erased type edition of status_error.
6  */
7  template <> class status_error<void> : public std::exception
8  {
9  protected:
10     ///! Constructs an instance. Not publicly available.
11     status_error() = default;
12     ///! Copy constructor. Not publicly available
13     status_error(const status_error &) = default;
14     ///! Move constructor. Not publicly available
15     status_error(status_error &&) = default;
16     ///! Copy assignment. Not publicly available
17     status_error &operator=(const status_error &) = default;
18     ///! Move assignment. Not publicly available
19     status_error &operator=(status_error &&) = default;
20     ///! Destructor. Not publicly available.
21     ~status_error() override = default;
22
23     virtual const status_code<void> &do_code() const noexcept = 0;
24
25 public:
26     ///! The type of the status domain
27     using domain_type = void;
28     ///! The type of the status code
29     using status_code_type = status_code<void>;
30
31 public:
32     ///! The erased status code which generated this exception instance.
33     const status_code<void> &code() const noexcept { return do_code(); }
34 };
35
36 /*! Exception type representing a thrown status_code
37 */
38 template <class DomainType> class status_error : public status_error<void>
39 {
40     status_code<DomainType> _code;
41     typename DomainType::string_ref _msgref;
42
43     virtual const status_code<void> &do_code() const noexcept override final { return _code; }
44
45 public:
46     ///! The type of the status domain
47     using domain_type = DomainType;
48     ///! The type of the status code

```

```

49     using status_code_type = status_code<DomainType>;
50
51     ///! Constructs an instance
52     explicit status_error(status_code<DomainType> code);
53
54     ///! Return an explanatory string
55     virtual const char *what() const noexcept override;
56
57     ///! Returns a reference to the code
58     const status_code_type &code() const &;
59     ///! Returns a reference to the code
60     status_code_type &code() &;
61     ///! Returns a reference to the code
62     const status_code_type &&code() const &&;
63     ///! Returns a reference to the code
64     status_code_type &&code() &&;
65 };

```

4.9 Generic error coding

```

1  ///! The generic error coding (POSIX)
2  enum class errc : int
3  {
4      success = 0,      ///!< This is new over std::errc
5      unknown = -1,    ///!< This is new over std::errc
6
7      address_family_not_supported = EAFNOSUPPORT,
8      address_in_use = EADDRINUSE,
9      address_not_available = EADDRNOTAVAIL,
10     already_connected = EISCONN,
11     argument_list_too_long = E2BIG,
12     argument_out_of_domain = EDOM,
13     bad_address = EFAULT,
14     bad_file_descriptor = EBADF,
15     bad_message = EBADMSG,
16     broken_pipe = EPIPE,
17     connection_aborted = ECONNABORTED,
18     connection_already_in_progress = EALREADY,
19     connection_refused = ECONNREFUSED,
20     connection_reset = ECONNRESET,
21     cross_device_link = EXDEV,
22     destination_address_required = EDESTADDRREQ,
23     device_or_resource_busy = EBUSY,
24     directory_not_empty = ENOTEMPTY,
25     executable_format_error = ENOEXEC,
26     file_exists = EEXIST,
27     file_too_large = EFBIG,
28     filename_too_long = ENAMETOOLONG,
29     function_not_supported = ENOSYS,
30     host_unreachable = EHOSTUNREACH,
31     identifier_removed = EIDRM,
32     illegal_byte_sequence = EILSEQ,
33     inappropriate_io_control_operation = ENOTTY,
34     interrupted = EINTR,

```

```

35     invalid_argument = EINVAL,
36     invalid_seek = ESPIPE,
37     io_error = EIO,
38     is_a_directory = EISDIR,
39     message_size = EMSGSIZE,
40     network_down = ENETDOWN,
41     network_reset = ENETRESET,
42     network_unreachable = ENETUNREACH,
43     no_buffer_space = ENOBUFS,
44     no_child_process = ECHILD,
45     no_link = ENOLINK,
46     no_lock_available = ENOLCK,
47     no_message = ENOMSG,
48     no_protocol_option = ENOPROTOOPT,
49     no_space_on_device = ENOSPC,
50     no_stream_resources = ENOSR,
51     no_such_device_or_address = ENXIO,
52     no_such_device = ENODEV,
53     no_such_file_or_directory = ENOENT,
54     no_such_process = ESRCH,
55     not_a_directory = ENOTDIR,
56     not_a_socket = ENOTSOCK,
57     not_a_stream = ENOSTR,
58     not_connected = ENOTCONN,
59     not_enough_memory = ENOMEM,
60     not_supported = ENOTSUP,
61     operation_cancelled = ECANCELED,
62     operation_in_progress = EINPROGRESS,
63     operation_not_permitted = EPERM,
64     operation_not_supported = EOPNOTSUPP,
65     operation_would_block = EWOULDBLOCK,
66     owner_dead = EOWNERDEAD,
67     permission_denied = EACCES,
68     protocol_error = EPROTO,
69     protocol_not_supported = EPROTONOSUPPORT,
70     read_only_file_system = EROFS,
71     resource_deadlock_would_occur = EDEADLK,
72     resource_unavailable_try_again = EAGAIN,
73     result_out_of_range = ERANGE,
74     state_not_recoverable = ENOTRECOVERABLE,
75     stream_timeout = ETIME,
76     text_file_busy = ETXTBSY,
77     timed_out = ETIMEDOUT,
78     too_many_files_open_in_system = ENFILE,
79     too_many_files_open = EMFILE,
80     too_many_links = EMLINK,
81     too_many_symbolic_link_levels = ELOOP,
82     value_too_large = EOVERFLOW,
83     wrong_protocol_type = EPROTOTYPE
84 };
85
86 //! A specialisation of 'status_error' for the generic code domain.
87 using generic_error = status_error<generic_code_domain>;
88 //! A constexpr source variable for the generic code domain, which is that of 'errc'
89 //! (POSIX). Returned by '_generic_code_domain::get()'.
90 constexpr _generic_code_domain generic_code_domain;

```

```

91 // Enable implicit construction of generic_code from errc
92 constexpr inline generic_code make_status_code(errc c) noexcept;

```

4.10 errored_status_code<DomainType>

```

1  /*! A 'status_code' which is always a failure. The closest equivalent to
2  'std::error_code', except it cannot be modified, and is templated.
3
4  Differences from 'status_code':
5
6  - Never successful (this contract is checked on construction, if fails then it
7  terminates the process).
8  - Is immutable.
9  */
10 template <class DomainType> class errored_status_code : public status_code<DomainType>
11 {
12     using _base = status_code<DomainType>;
13     using _base::clear;
14     using _base::success;
15
16 public:
17     /*! The type of the errored error code.
18     using typename _base::value_type;
19     /*! The type of a reference to a message string.
20     using typename _base::string_ref;
21
22     /*! Default constructor.
23     errored_status_code() = default;
24     /*! Copy constructor.
25     errored_status_code(const errored_status_code &) = default;
26     /*! Move constructor.
27     errored_status_code(errored_status_code &&) = default;
28     /*! Copy assignment.
29     errored_status_code &operator=(const errored_status_code &) = default;
30     /*! Move assignment.
31     errored_status_code &operator=(errored_status_code &&) = default;
32     ~errored_status_code() = default;
33
34     /*! Explicitly construct from any similar status code
35     constexpr explicit errored_status_code(const _base &o)
36         noexcept(std::is_nothrow_copy_constructible<_base>::value)
37         [[expects: o.failure() == true]];
38     /*! Explicitly construct from any similar status code
39     constexpr explicit errored_status_code(_base &&o)
40         noexcept(std::is_nothrow_move_constructible<_base>::value)
41         [[expects: o.failure() == true]];
42
43     /*! Implicit construction from any type where an ADL discovered
44     /*! 'make_status_code(T, Args ...)' returns a 'status_code'.
45     template <class T, class... Args, class MakeStatusCodeResult
46         = typename detail::safe_get_make_status_code_result<T, Args...>::type> // Safe ADL lookup of
47         make_status_code(), returns void if not found
48     requires(!std::is_same<typename std::decay<T>::type, errored_status_code>::value // not copy/move of
49             self

```



```

48     && !std::is_same<typename std::decay<T>::type, in_place_t>::value           // not
        in_place_t
49     && is_status_code<MakeStatusCodeResult>::value                           // ADL makes a
        status code
50     && std::is_constructible<errored_status_code, MakeStatusCodeResult>::value // ADLed status
        code is compatible
51 )
52 constexpr errored_status_code(T &&v, Args &&... args)
53     noexcept(noexcept(make_status_code(std::declval<T>(), std::declval<Args>()...)))
54     [[expects: make_status_code(std::forward<T>(v) /* unsafe? */, std::forward<Args>(args)...).failure
        () == true]];
55
56 /// Implicit construction from any 'quick_status_code_from_enum<Enum>' enumerated type.
57 template<class Enum, class QuickStatusCodeType
58     = typename quick_status_code_from_enum<Enum>::code_type>                // Enumeration has been
        activated
59 requires(std::is_constructible<errored_status_code, QuickStatusCodeType>::value) // Its status
        code is compatible
60 constexpr errored_status_code(Enum &&v)
61     noexcept(std::is_nothrow_constructible<errored_status_code, QuickStatusCodeType>::value)
62     [[expects: errored_status_code(QuickStatusCodeType(static_cast<Enum &&>(v))).failure() == true]];
63
64 /// Explicit in-place construction.
65 template <class... Args>
66 constexpr explicit errored_status_code(in_place_t /*unused */, Args &&... args)
67     noexcept(std::is_nothrow_constructible<value_type, Args &&...>::value)
68     [[expects: _base(std::forward<Args>(args)... /* unsafe? */).failure() == true]];
69
70 /// Explicit in-place construction from initializer list.
71 template <class T, class... Args>
72 constexpr explicit errored_status_code(in_place_t /*unused */, std::initializer_list<T> il, Args
    &&... args)
73     noexcept(std::is_nothrow_constructible<value_type, std::initializer_list<T>, Args &&...>::value)
74     [[expects: _base(il, std::forward<Args>(args)... /* unsafe? */).failure() == true]];
75
76 /// Explicit copy construction from a 'value_type'.
77 constexpr explicit errored_status_code(const value_type &v)
78     noexcept(std::is_nothrow_copy_constructible<value_type>::value)
79     [[expects: _base(v).failure() == true]];
80
81 /// Explicit move construction from a 'value_type'.
82 constexpr explicit errored_status_code(value_type &&v)
83     noexcept(std::is_nothrow_move_constructible<value_type>::value)
84     [[expects: _base(std::move(v) /* unsafe? */).failure() == true]];
85
86 /*! Explicit construction from an erased status code. Available only if
87 'value_type' is trivially destructible and 'sizeof(status_code) <= sizeof(status_code<erased<>>)'.
88 Does not check if domains are equal.
89 */
90 template <class ErasedType>
91 requires(detail::domain_value_type_erasure_is_safe<domain_type, erased<ErasedType>>::value)
92 constexpr explicit errored_status_code(const status_code<erased<ErasedType>> &v)
93     noexcept(std::is_nothrow_copy_constructible<value_type>::value)
94     [[expects: v.failure() == true]];
95
96 ///! Always false (including at compile time), as errored status codes are never successful.

```

```

97     constexpr bool success() const noexcept { return false; }
98     ///! Return a const reference to the 'value_type'.
99     constexpr const value_type &value() const &noexcept;
100 };

```

4.11 errored_status_code<erased<TRIVIAALLY_COPYABLE_OR_MOVE_BITCOPYING_TYPE>>

```

1  template <class ErasedType> class errored_status_code<erased<ErasedType>>
2      : public status_code<erased<ErasedType>>
3  {
4      using _base = status_code<erased<ErasedType>>;
5      using _base::success;
6
7  public:
8      using domain_type = typename _base::domain_type;
9      using value_type = typename _base::value_type;
10     using string_ref = typename _base::string_ref;
11
12     ///! Default construction to empty
13     errored_status_code() = default;
14     ///! Copy constructor
15     errored_status_code(const errored_status_code &) = default;
16     ///! Move constructor
17     errored_status_code(errored_status_code &&) = default;
18     ///! Copy assignment
19     errored_status_code &operator=(const errored_status_code &) = default;
20     ///! Move assignment
21     errored_status_code &operator=(errored_status_code &&) = default;
22     ~errored_status_code() = default;
23
24     ///! Explicitly construct from any similarly erased status code
25     constexpr explicit errored_status_code(const _base &o)
26         noexcept(std::is_nothrow_copy_constructible<_base>::value)
27         [[expects: o.failure() == true]];
28     ///! Explicitly construct from any similarly erased status code
29     constexpr explicit errored_status_code(_base &&o)
30         noexcept(std::is_nothrow_move_constructible<_base>::value)
31         [[expects: o.failure() == true]];
32
33     ///! Implicit copy construction from any other status code if its value type is
34     ///! trivially copyable, it would fit into our storage, and it is not an erased
35     ///! status code.
36     template <class DomainType>
37     requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value
38             && !detail::is_erased_status_code<status_code<typename std::decay<DomainType>::type>>::value
39             )
40     constexpr errored_status_code(const status_code<DomainType> &v) noexcept
41         [[expects: v.failure() == true]];
42
43     ///! Implicit copy construction from any other status code if its value type is
44     ///! trivially copyable, it would fit into our storage, and it is not an erased
45     ///! status code.
46     template <class DomainType>
47     requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value

```

```

47         && !detail::is_erased_status_code<status_code<typename std::decay<DomainType>::type>>::value
48     )
49
50 constexpr errored_status_code(const errored_status_code<DomainType> &v) noexcept;
51
52 /// Implicit move construction from any other status code if its value type is trivially copyable
53 /// or move relocating and it would fit into our storage
54 template <class DomainType>
55 requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value)
56 constexpr errored_status_code(status_code<DomainType> &&v) noexcept
57     [[expects: v.failure() == true]];
58
59 /// Implicit move construction from any other status code if its value type is trivially copyable
60 /// or move relocating and it would fit into our storage
61 template <class DomainType>
62 requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value)
63 constexpr errored_status_code(errored_status_code<DomainType> &&v) noexcept;
64
65 /// Implicit construction from any type where an ADL discovered 'make_status_code(T, Args ...)'
66 /// returns a 'status_code'.
67 template <class T, class... Args,
68         class MakeStatusCodeResult =
69             typename detail::safe_get_make_status_code_result<T, Args...>::type> // Safe ADL lookup of
70             make_status_code(), returns void if not found
71 requires(!std::is_same<typename std::decay<T>::type, errored_status_code>::value // not copy/move of
72         self
73         && !std::is_same<typename std::decay<T>::type, value_type>::value // not copy/move of
74         value type
75         && is_status_code<MakeStatusCodeResult>::value // ADL makes a
76         status code
77         && std::is_constructible<errored_status_code, MakeStatusCodeResult>::value)) // ADLed
78         status code is compatible
79 )
80 constexpr errored_status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::
81     declval<T>(), std::declval<Args>()...))) [[expects: make_status_code(std::forward<T>(v) /*
82     unsafe? */ , std::forward<Args>(args)...).failure() == true]];
83
84 /// Implicit construction from any 'quick_status_code_from_enum<Enum>' enumerated type.
85 template<class Enum,
86         class QuickStatusCodeType = typename quick_status_code_from_enum<Enum>::code_type>
87         // Enumeration has been activated
88 requires(std::is_constructible<errored_status_code, QuickStatusCodeType>::value) // Its status
89         code is compatible
90 constexpr errored_status_code(Enum &&v) noexcept(std::is_nothrow_constructible<status_code,
91     QuickStatusCodeType>::value);
92
93 /// Explicit copy construction from an unknown status code. Note that this will throw
94 /// an exception if its value type is not trivially copyable or would not
95 /// fit into our storage or the source domain's '_do_erased_copy()' refused the copy.
96 /// This function is not present if C++ exceptions are globally disabled.
97 explicit constexpr errored_status_code(const status_code<void> &v);
98
99 // errored_status_code(std::nothrow_t, const status_code<void> &v) is deliberately omitted,
100 // as empty errored_status_code's are not possible due to contract violation. One can
101 // use status_code's nothrow constructor, do a runtime check for emptiness, then implicitly
102 // construct an errored_status_code from that.

```

```

92  //! Always false (including at compile time), as errored status codes are never successful.
93  constexpr bool success() const noexcept { return false; }
94  //! Return the erased 'value_type' by value.
95  constexpr value_type value() const noexcept;
96  };

```

4.12 Errored status code comparisons

```

1  //! True if the status code's are semantically equal via 'equivalent()'.
2  template <class DomainType1, class DomainType2>
3  constexpr bool operator==(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
4      noexcept;
5  template <class DomainType1, class DomainType2>
6  constexpr bool operator==(const status_code<DomainType1> &a, const errored_status_code<DomainType2> &b)
7      noexcept;
8
9  //! True if the status code's are not semantically equal via 'equivalent()'.
10 template <class DomainType1, class DomainType2>
11 constexpr bool operator!=(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
12     noexcept;
13 template <class DomainType1, class DomainType2>
14 constexpr bool operator!=(const status_code<DomainType1> &a, const errored_status_code<DomainType2> &b)
15     noexcept;
16
17 //! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
18 template<class DomainType1, class T, class MakeStatusCodeResult
19     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
20     make_status_code(), returns void if not found
21     requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
22     status code
23     constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
24 template<class DomainType1, class T, class MakeStatusCodeResult
25     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
26     make_status_code(), returns void if not found
27     requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
28     status code
29     constexpr bool operator==(const errored_status_code<DomainType1> &a, const T &b);
30
31 //! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
32 template<class DomainType1, class T, class MakeStatusCodeResult
33     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
34     make_status_code(), returns void if not found

```

```

34 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
35 constexpr bool operator==(const T &a, const errored_status_code<DomainType1> &b);
36
37 //! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)'.
38 template<class DomainType1, class T, class MakeStatusCodeResult
39     = typename detail::safe_get_make_status_code_result<const T &::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
40 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
41 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
42 template<class DomainType1, class T, class MakeStatusCodeResult
43     = typename detail::safe_get_make_status_code_result<const T &::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
44 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
45 constexpr bool operator!=(const errored_status_code<DomainType1> &a, const T &b);
46
47 //! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)'.
48 template<class DomainType1, class T, class MakeStatusCodeResult
49     = typename detail::safe_get_make_status_code_result<const T &::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
50 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
51 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);
52 template<class DomainType1, class T, class MakeStatusCodeResult
53     = typename detail::safe_get_make_status_code_result<const T &::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
54 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
55 constexpr bool operator!=(const T &a, const errored_status_code<DomainType1> &b);
56
57 //! True if the status code's are semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(b)'.
58 template <class DomainType1, class T, class QuickStatusCodeType
59     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
60 >
61 constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
62 template <class DomainType1, class T, class QuickStatusCodeType
63     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
64 >
65 constexpr bool operator==(const errored_status_code<DomainType1> &a, const T &b);
66
67 //! True if the status code's are semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(a)'.
68 template <class DomainType1, class T, class QuickStatusCodeType
69     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
70 >
71 constexpr bool operator==(const T &a, const status_code<DomainType1> &b);
72 template <class DomainType1, class T, class QuickStatusCodeType
73     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
74 >
75 constexpr bool operator==(const T &a, const errored_status_code<DomainType1> &b);
76
77 //! True if the status code's are not semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(b)'.

```

```

78 template <class DomainType1, class T, class QuickStatusCodeType
79     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
80 >
81 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
82 template <class DomainType1, class T, class QuickStatusCodeType
83     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
84 >
85 constexpr bool operator!=(const errored_status_code<DomainType1> &a, const T &b);
86
87 //! True if the status code's are not semantically equal via 'equivalent()' to '
88     quick_status_code_from_enum<T>::code_type(a)'.
89 template <class DomainType1, class T, class QuickStatusCodeType
90     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
91 >
92 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);
93 template <class DomainType1, class T, class QuickStatusCodeType
94     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
95 >
96 constexpr bool operator!=(const T &a, const errored_status_code<DomainType1> &b);

```

4.13 Quick declaration of a new status code domain

```

1  //! A status code wrapping 'Enum' generated from 'quick_status_code_from_enum'.
2  template <class Enum>
3  using quick_status_code_from_enum_code = status_code<quick_status_code_from_enum_domain<Enum>>;
4
5  //! Defaults for an implementation of 'quick_status_code_from_enum<Enum>'
6  template <class Enum>
7  struct quick_status_code_from_enum_defaults
8  {
9      //! The type of the resulting code
10     using code_type = quick_status_code_from_enum_code<Enum>;
11     //! Used within 'quick_status_code_from_enum' to define a mapping of enumeration value with its
12         status code
13     struct mapping
14     {
15         //! The enumeration type
16         using enumeration_type = Enum;
17
18         //! The value being mapped
19         const Enum value;
20         //! A string representation for this enumeration value
21         const char *message;
22         //! A list of 'errc' equivalents for this enumeration value
23         const std::initializer_list<errc> code_mappings;
24     };
25     //! Used within 'quick_status_code_from_enum' to define mixins for the status code wrapping 'Enum'
26     template <class Base> struct mixin : Base
27     {
28         using Base::Base;
29     };
30 };
31
32 template <class Enum>

```

```

32 constexpr _quick_status_code_from_enum_domain<Enum> quick_status_code_from_enum_domain = {};
33
34 namespace mixins
35 {
36     template <class Base, class Enum> struct mixin<Base, _quick_status_code_from_enum_domain<Enum>> :
37         public quick_status_code_from_enum<Enum>::template mixin<Base>
38     {
39         using quick_status_code_from_enum<Enum>::template mixin<Base>::mixin;
40     } // namespace mixins

```

4.14 OS specific and common codes

```

1  // Lets you retrieve the 'std::error_category' from a 'std_error_code'
2  namespace mixins
3  {
4      template <class Base> struct mixin<Base, _std_error_code_domain> : public Base
5      {
6          using Base::Base;
7
8          //! Implicit constructor from a 'std::error_code'
9          inline mixin(std::error_code ec);
10
11         //! Returns the error code category
12         inline const std::error_category &category() const noexcept;
13     };
14 } // namespace mixins
15
16 //! A wrapper of 'std::error_code'. This passes through the original
17 //! error code's category during comparisons. This can't be an
18 //! 'errored_status_code', as 'std::error_code' can be null.
19 using std_error_code = status_code<error_code_domain<std::error_code, detail::make_std_categories>>;
20
21
22 // Provides static function 'posix_code::current()'
23 namespace mixins
24 {
25     template <class Base> struct mixin<Base, _posix_code_domain> : public Base
26     {
27         using Base::Base;
28
29         //! Returns a 'posix_code' for the current value of 'errno'.
30         static posix_code current() noexcept;
31     };
32 } // namespace mixins
33
34 //! A POSIX error code, those returned by 'errno'.
35 using posix_code = status_code<posix_code_domain>;
36 //! A specialisation of 'errored_status_code' for the POSIX error code domain.
37 using posix_error = errored_status_code<posix_code_domain>;
38
39
40 // Lets you question a 'http_status_code' for its characteristics
41 namespace mixins

```

```

42 {
43     template <class Base> struct mixin<Base, _http_status_code_domain> : public Base
44     {
45         using Base::Base;
46
47         ///! True if the HTTP status code is informational
48         inline bool is_http_informational() const noexcept;
49         ///! True if the HTTP status code is successful
50         inline bool is_http_success() const noexcept;
51         ///! True if the HTTP status code is redirection
52         inline bool is_http_redirection() const noexcept;
53         ///! True if the HTTP status code is client error
54         inline bool is_http_client_error() const noexcept;
55         ///! True if the HTTP status code is server error
56         inline bool is_http_server_error() const noexcept;
57     };
58 } // namespace mixins
59
60 ///! A HTTP status code.
61 using http_status_code = status_code<_http_status_code_domain>;
62 ///! A HTTP status code.
63 using http_status_error = errored_status_code<_http_status_code_domain>;
64
65
66 ///! A getaddrinfo error code, those returned by 'getaddrinfo()'.
67 using getaddrinfo_code = status_code<_getaddrinfo_code_domain>;
68 ///! A specialisation of 'errored_status_code' for the getaddrinfo code domain.
69 using getaddrinfo_error = errored_status_code<_getaddrinfo_code_domain>;
70
71
72 #ifndef _WIN32
73 // Provides static function 'win32_code::current()'
74 namespace mixins
75 {
76     template <class Base> struct mixin<Base, _win32_code_domain> : public Base
77     {
78         using Base::Base;
79
80         ///! Returns a 'win32_code' for the current value of 'GetLastError()'.
81         static inline win32_code current() noexcept;
82     };
83 } // namespace mixins
84
85 ///! (Windows only) A Win32 error code, those returned by 'GetLastError()'.
86 using win32_code = status_code<_win32_code_domain>;
87 ///! (Windows only) A specialisation of 'errored_status_code' for the Win32 error code domain.
88 using win32_error = errored_status_code<_win32_code_domain>;
89
90
91 ///! (Windows only) A NT error code, those returned by NT kernel functions.
92 using nt_code = status_code<_nt_code_domain>;
93 ///! (Windows only) A specialisation of 'errored_status_code' for the NT error code domain.
94 using nt_error = errored_status_code<_nt_code_domain>;
95
96
97 /*! (Windows only) A COM error code. Note semantic equivalence testing is only

```



```

98 implemented for 'FACILITY_WIN32' and 'FACILITY_NT_BIT'. As you can see at
99 [https://blogs.msdn.microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/](https://blogs.msdn.
    microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/),
100 there are an awful lot of COM error codes, and keeping mapping tables for all of
101 them would be impractical (for the Win32 and NT facilities, we actually reuse the
102 mapping tables in 'win32_code' and 'nt_code'). You can, of course, inherit your
103 own COM code domain from this one and override the '_equivalent()' function
104 to add semantic equivalence testing for whichever extra COM codes that your
105 application specifically needs.
106 */
107 using com_code = status_code<com_code_domain>;
108 //! (Windows only) A specialisation of 'errored_status_code' for the COM error code domain.
109 using com_error = errored_status_code<com_code_domain>;
110 #endif // _WIN32

```

4.15 Erased system code, and proposed `std::error` object

```

1  /*! An erased-mutable status code suitably large for all the system codes
2  which can be returned on this system.
3
4  For Windows, these might be:
5
6      - 'com_code' ('HRESULT') [you need to include "com_code.hpp" explicitly for this]
7      - 'nt_code' ('LONG')
8      - 'win32_code' ('DWORD')
9
10 For POSIX, 'posix_code' and 'getaddrinfo_code' is possible.
11
12 You are guaranteed that 'system_code' can be transported by the compiler
13 in exactly two CPU registers.
14 */
15 using system_code = status_code<erased<intptr_t>>;
16
17 /*! A utility function which returns the closest matching system_code to a supplied
18 exception ptr.
19 */
20 inline system_code system_code_from_exception(std::exception_ptr &&ep = std::current_exception(),
21     system_code not_matched = generic_code(errc::resource_unavailable_try_again)) noexcept;
22
23
24 /*! An errored 'system_code' which is always a failure. The closest equivalent to
25 'std::error_code', except it cannot be null and cannot be modified.
26
27 This refines 'system_code' into an 'error' object meeting the requirements of
28 [P0709 Zero-overhead deterministic exceptions](https://wg21.link/P0709).
29
30 Differences from 'system_code':
31
32 - Always a failure (this is checked at construction, and if not the case,
33 the program is terminated as this is a logic error)
34 - Is immutable.
35
36 As with 'system_code', it remains guaranteed to be two CPU registers in size,
37 and trivially copyable.

```

```

38 */
39 using error = errored_status_code<erased<system_code::value_type>>;

```

4.16 ostream printing support

```

1  /*! Print the status code to a 'std::ostream &'.
2  Requires that 'DomainType::value_type' implements an 'operator<<' overload for 'std::ostream'.
3  */
4  template <class DomainType>
5  requires(std::is_same<std::ostream, typename std::decay<decltype(std::declval<std::ostream>()) << std::
6  declval<typename status_code<DomainType>::value_type>())>::type>::value)
7  std::ostream &operator<<(std::ostream &s, const status_code<DomainType> &v);
8
9  /*! Print a status code domain's 'string_ref' to a 'std::ostream &'.
10 */
11 std::ostream &operator<<(std::ostream &s, const status_code_domain::string_ref &v);
12
13 /*! Print the erased status code to a 'std::ostream &'.
14 */
15 template <class ErasedType>
16 std::ostream &operator<<(std::ostream &s, const status_code<erased<ErasedType>> &v);
17
18 /*! Print the generic code to a 'std::ostream &'.
19 */
20 std::ostream &operator<<(std::ostream &s, const generic_code &v);

```

4.17 status code ptr

```

1  /*! Make an erased status code which indirects to a dynamically allocated status code.
2  This is useful for shoehorning a rich status code with large value type into a small
3  erased status code like 'system_code', with which the status code generated by this
4  function is compatible. Note that this function can throw due to 'bad_alloc'.
5  */
6  template <class T>
7  requires(is_status_code<T>::value)
8  status_code<erased<typename std::add_pointer<typename std::decay<T>::type>::type>>
9  make_status_code_ptr(T &&v);
10
11 /*! If a status code refers to a 'status_code_ptr' which indirects to a status
12 code of type 'StatusCode', return a pointer to that 'StatusCode'. Otherwise return null.
13 */
14 template <class StatusCode, class U>
15 requires(is_status_code<StatusCode>::value)
16 StatusCode *get_if(status_code<erased<U>> *v) noexcept;
17
18 /// \overload Const overload
19 template <class StatusCode, class U>
20 requires(is_status_code<StatusCode>::value)
21 const StatusCode *get_if(const status_code<erased<U>> *v) noexcept;
22

```

```

23
24  /*! If a status code refers to a 'status_code_ptr' which indirects to a status
25  code of type 'StatusCode', return a reference to that 'StatusCode'. Otherwise throw
26  'bad_status_ptr_access'.
27  */
28  template <class StatusCode, class U>
29  requires(is_status_code<StatusCode>::value)
30  StatusCode &get(status_code<erased<U>> &v);
31
32  ///! \overload Const overload
33  template <class StatusCode, class U>
34  requires(is_status_code<StatusCode>::value)
35  const StatusCode &get(const status_code<erased<U>> &v);
36
37
38  /*! If a status code refers to a 'status_code_ptr', return the id of the erased
39  status code's domain. Otherwise return a meaningless number.
40  */
41  template <class U>
42  typename status_code_domain::unique_id_type get_id(const status_code<erased<U>> &v) noexcept;

```

5 Design decisions, guidelines and rationale

These are copied from [P0824] *Summary of SG14 discussion on <system_error>* for your information.

5.1 Do not cause `#include <string>`

`<system_error>`, on all the major STL implementations, includes `<string>` as `std::error_code::message()`, amongst other facilities, returns a `std::string`. `std::string`, in turn, drags in the STL allocator machinery and a fair few algorithms and other headers.

Bringing in so much extra stuff is a showstopper for the use of `std::error_code` in the global APIs of very large C++ code bases due to the effects on build and link times. As much as C++ Modules may, or may not, fix this some day, adopting `std::error_code` – which is highly desirable to large C++ code bases which globally disable C++ exceptions such as games – is made impossible. Said users end up having to locally reinvent a near clone of `std::error_code`, but one which doesn't use `std::string`, which is unfortunate.

Moreover, because `<stdexcept>` must include `<system_error>`, and many otherwise very simple STL facilities such as `<array>`, `<complex>`, `<iterator>` or `<optional>` must include `<stdexcept>`, we end up dragging in `<string>` and the STL allocator machinery when including those otherwise simple and lightweight STL headers for no good purpose other than that `std::error_code::message()` returns a `std::string`! That deprives very large C++ code bases of being able to use `std::optional<T>` and other such vocabulary types in their global headers.

Hence, this implicit dependency of `<system_error>` on `<string>` contravenes [P2000]'s admonition *'Note that the cost of compilation is among the loudest reasonable complaints about C++ from its*

users’

It also breaks the request ‘*make C++ easier to use and more effective for large and small embedded systems*’ by making a swathe of C++ library headers not [P0829] *Freestanding C++* compatible.

It is trivially easy to fix: stop using `std::string` to return textual representation of codes. This proposed design uses a `string_ref` instead, this is a potentially reference counted handle to a string. It is extremely lightweight, freestanding C++ compatible, and drags in no unnecessary headers.

5.2 All constexpr sourcing, construction and destruction

`<system_error>` was designed before constexpr entered the language, and many operations which ought to be constexpr for such a simple and low-level facility are not. Simple things like the `std::error_code` constructor is not constexpr, bigger things like `std::error_category` are not constexpr, and far more importantly the global source of error code categories is not constexpr, forcing the compiler to emit a magic static initialisation fence, which introduces significant added code bloat as magic fences cannot be elided by the optimiser.

The proposed replacement makes everything which can be constexpr be just that. If it cannot be constexpr, it is literal or trivial to the maximum extent possible. Empirical testing in real world code bases has found excellent effects on the density of assembler generated, with recent GCCs and clangs, almost all of the time the code generated with the replacement design is as optimal as a human assembler writer might write.

5.3 Header only libraries can now safely define custom code categories

Something probably unanticipated at the time of the design of `<system_error>` is that bespoke `std::error_category` implementations are unsafe in header only libraries. This has caused significant, and usually unpleasant, surprise in the C++ user base.

The problem stems from the comparison of `std::error_category` implementations which is *required* by the C++ standard to be a comparison of address of instance. When comparing an error code to an error condition, the `std::error_category::equivalent()` implementation compares the input error code’s category against a list of error code categories known to it in order to decide upon equivalence. This is by address of instance.

Header only libraries must use Meyer singletons to implement the source of the custom `std::error_category` implementation i.e.

```
1 inline const my_custom_error_category &custom_category()  
2 {  
3     static my_custom_error_category v;  
4     return v;  
5 }
```

Ordinarily speaking, the linker would choose one of these inline function implementations, and thus `my_custom_error_category` gets exactly one instance, and thus one address in the final executable. All would therefore seem good.

Problems begin when a user uses the header only library inside a shared library. Now there is a single instance of the inline function *per shared library*, not per final executable. It is not uncommon for users to use more than one shared library, and thus multiple instances of the inline function come into existence. You now get the unpleasant situation where there are multiple singletons in the process, each with a different address, despite being the same error code category. Comparisons between error codes and categories thus subtly break in a somewhat chance based, hard to debug, way¹.

Those bitten by this ‘feature’ tend to be quite bitter about it. This author is one of those embittered. He has met others who have been similarly bitten through the use of ASIO and the Boost C++ Libraries. It’s a niche problem, but one which consumes many days of very frustrating debugging for the uninitiated.

The proposed design makes error category sources all-constexpr as well as error code construction. This is incompatible with singletons, so the proposed design does away with the need for singleton sources entirely in favour of stateless code domains with a static random unique 64-bit id, of which there can be arbitrarily many instantiated at once, and thus the proposed design is safe for use in header only libraries.

In case there is concern of collision in a totally random unique 64 bit id, here are the number of random 64-bit numbers needed in the same process space for various probabilities of collision (note that 10e15 is the number of bits which a hard drive guarantees to return without mistake):

Probability of collision	10e-15	10e-12	10e-9	10e-6	10e-3 (0.1%)	10e-2 (1%)
Random 64-bit numbers needed	190	6100	190,000	6,100,000	190,000,000	610,000,000

5.4 No more `if(!ec)...`

`std::error_code` provides a boolean test. The correct definition for the meaning of the boolean test is ‘is the value in this error code all bits zero, ignoring the category?’. It does **not** mean ‘is there no error?’.

This may seem like an anodyne distinction, but it causes real confusion. During a discussion on the Boost C++ Libraries list regarding this issue, multiple opinions emerged over whether this was ambiguous, whether it would result in bugs, whether it was serious, whether programmers who wrote the code assuming the latter were the ones at fault, or whether it was the meaning of the boolean test. No resolution was found.

All this suggests to SG14 that there is unhelpful ambiguity which we believe can never lead to better quality software, so we have removed the boolean test in the proposed design. Developers must now be clear as to exactly what they mean: `if(ec.success())...`, `if(ec.failure())...` and so on.

¹Do inline variables help? Unfortunately not. They suffer from the same problem of instance duplication when used in shared libraries. This is because standard C++ code has no awareness of shared libraries.

5.5 No more filtering codes returned by system APIs

Because `std::error_code` treats all bits zero values specially, and its boolean test does not consider category at all, when constructing error codes after a syscall, one must inevitably add some logic which performs a local check of whether the system returned code is a failure or not, and only then follow the error path.

This is fine for a lot of use cases, but many platforms, and indeed third party libraries, like to return success-with-information or success-with-warning codes. The current `<system_error>` does not address the possibility of multiple success codes being possible, nor that there is any success code other than all bits zero.

It also forces the program code which constructs the system code into an error code to be aware of implementation details of the source of the code in order to decide whether it is a failure or not. That is usually the case, but is not always the case. For where it is not the case, forcing this on users breaks clean encapsulation.

The proposed redesign accepts unfiltered and unmodified codes from any source. The category – called a *domain* in this proposal – interprets codes of any form of success or failure. Users can always safely construct a `status_code` (in this proposal, not [P0262]’s `status_value`) without knowing anything about the implementation details of its source. No one value is treated specially from any other.

5.6 All comparisons between codes are now semantic, not literal

Even some members of WG21 get the distinction between `std::error_code` and `std::error_condition` incorrect. That is because they appear to be almost the same thing, the same design, same categories, with only a vague documentation that one is to be used for system-specific codes and the other for non-system-specific codes.

This leads to an unnecessarily steep learning curve for the uninitiated, confusion amongst programmers reading code, incorrect choice of `std::error_condition` when `std::error_code` was meant, surprise when comparisons between codes and conditions are semantic not literal, and more of that general ambiguity and confusion we mentioned earlier.

The simple solution is to do away with all literal comparison entirely. Comparisons of `status_code` are **always** semantic. If the user really does want a literal comparison, they can manually compare domain and values by hand. Almost all of the time they actually want semantic comparison, and thus `operator ==`’s non-regular semantic comparison is exactly right.

5.7 `std::error_condition` is removed entirely

As comparisons are now always semantic between `status_code`’s, there is no longer any need for a distinction between `std::error_code` and `std::error_condition`. We therefore simplify the situation by removing any notion of `std::error_condition` altogether.

5.8 `status_code`'s value type is set by its domain

`std::error_code` hard codes its value to an `int`, which is problematic for third party error coding schemes which use a `long`, or even an `unsigned int`. `status_code<DomainType>` sets its `value_type` to be `DomainType::value_type`. Thus if you define your own domain type, its value type can be any type you like, including a structure or class.

This enables *payload* to be transmitted with your status code e.g. if the status code represents a failure in the filesystem, the payload might contain the path of a relevant file. It might contain the stack backtrace of where a failure or warning occurred, a `std::exception_ptr` instance, or anything else you might like.

We make great use of this domain definable value type facility to wrap up all possible `std::error_code`'s into status codes via a code domain whose value type is a `std::error_code`. This enables complete participation of any existing error code scheme within the proposed status code scheme.

5.9 `status_code<DomainType>` is type erasable

`status_code<DomainType>` can be type erased into a `status_code<void>` which is an immutable, unrelocatable, uncopyable type suitable for passing around by const lvalue reference only. This allows non-templated code to work with arbitrary, unknown, `status_code<DomainType>` instances. One may no longer retrieve their value obviously, but one can still query them for whether they represent success or failure, or for a textual message representing their value, and so on.

If, and only if, `DomainType::value_type` and some type `U` are `TriviallyCopyable` and the size of `DomainType::value_type` is less than or equal to size of `U`, an additional type erasure facility becomes available, that of `status_code<erased<U>>`. Unlike `status_code<void>`, this type erased form is copyable which is safe as `DomainType::value_type` and `U` are `TriviallyCopyable`, and are therefore both copyable as if via `memcpy()`.

This latter form of type erasure is particularly powerful. It allows one to define some global `status_code<erased<U>>` which is common to all code: `status_code<erased<intptr_t>>` would be a very portable choice². Individual components may work in terms of `status_code<LocalErrorType>`, but all public facing APIs may return only the global `status_code<erased<intptr_t>>`. This facility thus allows any arbitrary `LocalErrorType` to be returned, unmodified, *with value semantics* through code which has no awareness of it. The only conditions are that `LocalErrorType` is trivially copyable, and is not bigger than the erased `intptr_t` type.

5.10 More than one 'system' error coding domain: `system_code`

`std::system_category` assumes that there is only one 'system' error coding, something not even true on POSIX (note that POSIX's error coding is always a subset of the POSIX implementation's

²Why? On x64 with SysV calling convention, a trivially copyable object no more than two CPU registers of size will be returned from functions via CPU registers, saving quite a few CPU cycles. AArch64 will return trivially copyable objects of up to 64 bytes via CPU registers!

error coding), let alone elsewhere, especially on Microsoft Windows where at least four primary system error coding schemes exist: (i) POSIX `errno` (ii) Win32 `GetLastError()` (iii) NT kernel `NTSTATUS` (iv) COM/WinRT/DirectX `HRESULT`.

The proposed library makes use of the `status_code<erased<U>>` facility described in the previous section to define a type alias `system_code` to a type erased status code sufficiently large enough to carry any of the system error codings on the current platform. This allows code to use the precise error code domain for the system failure in question, and to return it type erased in a form perfectly usable by external code, which need neither know nor care that the failure stemmed originally from COM, or Win32, or POSIX. All that matters is that the status code semantically compares true to say `std::errc::no_such_file_or_directory`.

5.11 `std::errc` gets its own code domain `generic_code`, eliminating `std::error_condition`

Similar, but orthogonal, to `system_code` is `generic_code` which has a value type of the strongly typed enum `std::errc`. Codes in the generic code domain become the ‘portable error codes’ formerly represented by `std::error_condition` in that they act as semantic comparator of last resort.

Generic codes allow one to write code which semantically compares success or failure to the standard failure reasons defined by POSIX. This allows one to write portable code which works independent of platform and implementation.

6 Technical specifications

No Technical Specifications are involved in this proposal.

7 Frequently asked questions

7.1 Implied in this design is that code domains must do nothing in their constructor and destructors, as multiple instances are permitted and both must be trivial and constexpr. How then can dynamic per-domain initialisation be performed e.g. setting up at run time a table of localised message strings?

The simplest is to use statically initialised local variables, though be aware that it is always legal to use status code from within static initialisation and finalisation, so you need to lazily construct any tables on first use and never deallocate. Slightly more complex is to use the domain’s `string_ref` instances to keep a reference count of the use of the code domain, when all `string_ref` instances are destroyed, it is safe to deallocate any per-domain data.

7.2 Move only `std::error`?

For some reason people object to move-only `error`, which is the erased form of status code able to transport payloads of unknown type. This design decision needs to therefore be explained.

C++ is a strongly typed language, this allows its optimiser to make strong assumptions about code e.g. it can be copied. Erasing the type reduces the assumptions which can be made by the compiler, requiring decisions to be taken by run-time code instead. This is exactly the case with erased status code, which must be move only in case the erased type they transport is move only e.g. if the payload were a unique ptr, it needs to be move only. And a move only erased status code can transport copyable types just fine, but the compiler can't know that from the outside.

There is a `.clone()` member function on erased status codes. This asks the code's domain to clone the erased type, as it knows what that type and its payload is. It may refuse by throwing an exception. Because of the type erasure, this can never be a trivial operation, which is exactly why copy construction is disabled by default and you must explicitly opt into the cost of the clone via `.clone()`.

8 Addendum: Boost's `result<T>` type

I wish to state that I would **greatly** prefer if [P0709] *Zero overhead deterministic exceptions* were standardised instead of `result<T>`. It is superior in every way, and much less verbose and fiddly to work with, than functions returning Result types. We also get the ability to have constructors fail deterministically, instead of the usual two-stage static initialisation tricks, or having to use free functions as constructors.

But equally we cannot hold up standards proposals which require deterministic exceptions until they enter the language, which may be never. So I reluctantly propose a subset of `result<T>` derived from (Boost.)Outcome and Boost.System³, hardcoded around `E` being always an `error`.

`result<T>` looks annoyingly different to `optional<T>` and `expected<T, error>` in that it has a `variant`-modelled constructor interface, and clearly separated wide and narrow contract observers like variant does. This is quite different to the `optional`-modelled constructor interface of `Expected`, which has mixed wide and narrow contract observers. Part of the difference stems from Result being designed after much use experience of Optional by the Boost developer community. Part of the difference stems from Variant by then entering the standard. And part of the difference is because of the hard coding of the alternative type to `error`. All this led to the eventual choosing of the presented design, instead of `Expected`'s design, which occurred after one of the most voluminous peer reviews Boost has ever undertaken.

Outcome's result type never has a valueless state, however Boost.System's result *does* have a valueless state. It may, or may not, use union storage (Outcome's current implementation uses struct storage when `E` is `error`, as it has lower impact on compile times). Because type erased `error` is move only, `result<T>` is always move only (if you want a copy, call `.clone()`). Result has a bitcopying

³Since R3 of this paper, Boost.System has reimplemented this paper's proposed `result<T>`. See https://www.boost.org/doc/libs/1_80_0/libs/system/doc/html/system.html#ref_resultt_e.

move constructor if `T` is trivially copyable or move bitcopying (see [P1029] *move* = *bitcopies*). Comparisons deliberately exclude anything except equality, this prevents the perplexing surprise which otherwise occur with types which implicitly construct from values they can transport.

```

1  /*! \brief Exception type representing the failure to retrieve an error.
2  */
3  class bad_result_access;
4
5  /*! \class result
6  \brief A 'result<T>' type with its error type hardcoded to 'error'.
7
8  This may, or may not, have union storage. It must never have a valueless state.
9  */
10 template <class T>
11 requires(!std::is_reference_v<T> && !std::is_array_v<T> && !std::is_same_v<T, error>)
12 class result
13 {
14 public:
15     ///! The value type
16     using value_type = T;
17     ///! The error type
18     using error_type = error;
19
20     ///! Used to rebind result types
21     template <class U> using rebind = result<U>;
22
23 public:
24     ///! Default constructor is disabled
25     result() = delete;
26     ///! Copy constructor is disabled
27     result(const result &) = delete;
28     ///! Move constructor. See P1029 move = bitcopies.
29     result(result &&) = bitcopies(auto);
30     ///! Copy assignment is disabled
31     result &operator=(const result &) = delete;
32     ///! Move assignment
33     result &operator=(result &&) = default;
34     ///! Destructor
35     ~result() = default;
36
37     ///! Implicit result converting move constructor
38     template <class U>
39     requires(std::is_convertible_v<U, T>)
40     constexpr result(result<U> &&o) noexcept(std::is_nothrow_constructible_v<T, U>);
41
42     ///! Implicit result converting copy constructor
43     template <class U>
44     requires(std::is_convertible_v<U, T>)
45     constexpr result(const result<U> &o) noexcept(std::is_nothrow_constructible_v<T, U>);
46
47     ///! Explicit result converting move constructor
48     template <class U>
49     requires(std::is_constructible_v<T, U>)
50     constexpr explicit result(result<U> &&o) noexcept(std::is_nothrow_constructible_v<T, U>);
51
52     ///! Explicit result converting copy constructor

```

```

53 template <class U>
54 requires(std::is_constructible_v<T, U>)
55 constexpr explicit result(const result<U> &o) noexcept(std::is_nothrow_constructible_v<T, U>);
56
57 /* Slightly wider constructors than std::variant<error, T> would have. */
58
59 //! Implicit in-place converting value constructor
60 template<class Arg1, class... Args>
61 requires(! (std::is_constructible_v<value_type, Arg1, Args...>
62             && std::is_constructible_v<error_type, Arg1, Args...>) //
63           &&std::is_constructible_v<value_type, Arg1, Args...>)
64 constexpr variant(Arg1 &&arg1, Args &&... args) noexcept(...);
65
66 //! Implicit in-place converting error constructor
67 template<class Arg1, class... Args>
68 requires(! (std::is_constructible_v<value_type, Arg1, Args...>
69             && std::is_constructible_v<error_type, Arg1, Args...>) //
70           &&std::is_constructible_v<error_type, Arg1, Args...>)
71 constexpr variant(Arg1 &&arg1, Args &&... args) noexcept(...);
72
73 //! Explicit in-place constructor for either value or error
74 template< class Arg, class... Args >
75 constexpr explicit result(std::in_place_type_t<Arg>, Args&&... args) noexcept(...);
76
77 //! Explicit in-place constructor for either value or error
78 template< class Arg, class... Args >
79 constexpr explicit result(std::in_place_type_t<Arg>, Args&&... args) noexcept(...);
80
81 //! Explicit in-place constructor for either value or error
82 template< class Arg1, class Arg2, class... Args >
83 constexpr explicit result(std::in_place_type_t<Arg1>,
84                          std::initializer_list<Arg2> il, Args&&... args) noexcept(...);
85
86 //! Implicit construction from any type where an ADL discovered 'make_status_code(T, Args ...)'
87 //! returns a 'status_code'.
88 template <class U, class... Args>
89 requires(/* safe ADL lookup of make_status_code*/)
90 constexpr result(U &&v, Args &&... args) noexcept(noexcept(make_status_code(std::declval<U>()), std::declval<Args>()...));
91
92 //! Swap with another result
93 constexpr void swap(result &o) noexcept(...);
94
95 //! Clone the result
96 constexpr result clone() const;
97
98 //! True if result has a value
99 constexpr bool has_value() const noexcept;
100
101 //! True if result has a value
102 explicit operator bool() const noexcept;
103
104 //! True if result has an error
105 constexpr bool has_error() const noexcept;
106
107 //! Accesses the value if one exists, else calls '.error().throw_exception()'.

```

```

108 constexpr value_type_if_enabled &value() &;
109
110 //! Accesses the value if one exists, else calls '.error().throw_exception()'.
111 constexpr const value_type_if_enabled &value() const &;
112
113 //! Accesses the value if one exists, else calls '.error().throw_exception()'.
114 constexpr value_type_if_enabled &&value() &&;
115
116 //! Accesses the value if one exists, else calls '.error().throw_exception()'.
117 constexpr const value_type_if_enabled &&value() const &&;
118
119 //! Accesses the error if one exists, else throws 'bad_result_access'.
120 constexpr error_type &error() &;
121
122 //! Accesses the error if one exists, else throws 'bad_result_access'.
123 constexpr const error_type &error() const &;
124
125 //! Accesses the error if one exists, else throws 'bad_result_access'.
126 constexpr error_type &&error() &&;
127
128 //! Accesses the error if one exists, else throws 'bad_result_access'.
129 constexpr const error_type &&error() const &&;
130
131
132 //! Accesses the value, being UB if none exists
133 constexpr value_type_if_enabled &assume_value() & noexcept;
134
135 //! Accesses the error, being UB if none exists
136 constexpr const value_type_if_enabled &assume_value() const & noexcept;
137
138 //! Accesses the error, being UB if none exists
139 constexpr value_type_if_enabled &&assume_value() && noexcept;
140
141 //! Accesses the error, being UB if none exists
142 constexpr const value_type_if_enabled &&assume_value() const && noexcept;
143
144
145 //! Accesses the error, being UB if none exists
146 constexpr error_type &assume_error() & noexcept;
147
148 //! Accesses the error, being UB if none exists
149 constexpr const error_type &assume_error() const & noexcept;
150
151 //! Accesses the error, being UB if none exists
152 constexpr error_type &&assume_error() && noexcept;
153
154 //! Accesses the error, being UB if none exists
155 constexpr const error_type &&assume_error() const && noexcept;
156 };
157
158 //! True if the two results compare equal. Available only if T can be compared to U.
159 template <class T, class U>
160 requires(...)
161 constexpr inline bool operator==(const result<T> &a, const result<U> &b) noexcept;
162
163 //! True if the two results compare unequal. Available only if T can be compared to U.

```

```
164 template <class T, class U>  
165 requires(...)   
166 constexpr inline bool operator!=(const result<T> &a, const result<U> &b) noexcept;
```

9 Acknowledgements

Thanks to Ben Craig, Arthur O'Dwyer and Vicente J. Botet Escriba for their comments and feedback.

Thanks to Jesse Towner for walking through the code and pointing out lots of small mistakes, as well as contributing a patchset fixing memory leaks, improving constexpr type erasure, and other implementation improvements.

10 References

- [N2066] Beman Dawes,
TR2 Diagnostics Enhancements
<https://wg21.link/N2066>
- [P0262] Lawrence Crowl, Chris Mysen,
A Class for Status and Optional Value
<https://wg21.link/P0262>
- [P0709] Herb Sutter,
Zero-overhead deterministic exceptions: Throwing values
<https://wg21.link/P0709>
- [P0824] O'Dwyer, Bay, Holmes, Wong, Douglas,
Summary of SG14 discussion on <system_error>
<https://wg21.link/P0824>
- [P0829] Ben Craig,
Freestanding proposal
<https://wg21.link/P0829>
- [P1029] Douglas, Niall
move = bitcopies
<https://wg21.link/P1029>
- [P1031] Douglas, Niall
Low level file i/o library
<https://wg21.link/P1031>
- [P1095] Douglas, Niall
Zero overhead deterministic failure – A unified mechanism for C and C++
<https://wg21.link/P1095>

- [P1144] O'Dwyer, Arthur
Object relocation in terms of move plus destroy
<https://wg21.link/P1144>
- [P1195] Dimov, Peter
Making `<system_error>` constexpr
<https://wg21.link/P1195>
- [P1196] Dimov, Peter
Value-based `std::error_category` comparison
<https://wg21.link/P1196>
- [P1197] Dimov, Peter
A non-allocating overload of `error_category::message()`
<https://wg21.link/P1197>
- [P1198] Dimov, Peter
Adding `error_category::failed()`
<https://wg21.link/P1198>
- [P1631] Douglas, Niall and Steagall, Bob
Object detachment and attachment
<https://wg21.link/P1631>
- [P1883] Douglas, Niall
`file_handle` and `mapped_file_handle`
<https://wg21.link/P1883>
- [P2000] H. Hinnant, B. Stroustrup, D. Vandevor, M. Wong,
Direction for ISO C++
<http://wg21.link/P2000>
- [P2170] Salvia, Charles
Feedback on implementing the proposed `std::error` type
<https://wg21.link/P2170>
- [1] *Boost.Outcome*
Douglas, Niall and others
<https://ned14.github.io/outcome/>
- [2] *stl-header-heft github analysis project*
Douglas, Niall
<https://github.com/ned14/stl-header-heft>